



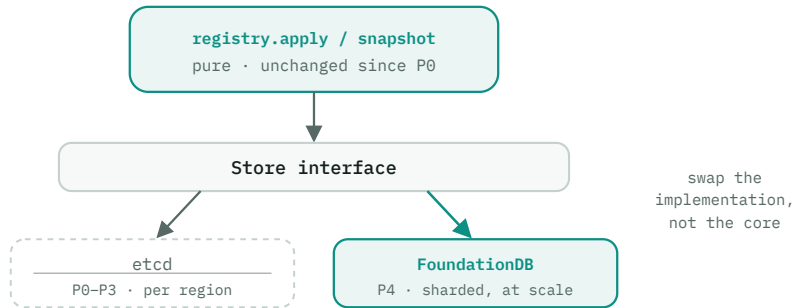
— COMPONENT DESIGN • PHASE 4

The phase where the boundary pays off.

P4 takes the swarm to a million machines — a **FoundationDB** registry, metric **rollups**, **backpressure**, and **rolling upgrades**. Almost none of it touches a core: the decisions were drawn pure back in P0–P3, so going to scale is a **shell** concern. That's the FCIS line earning its keep.

01 Scaling is a shell change

The store the registry writes to, the transport it rides on, the metrics it emits — all of that lives in the **shell**. The *decisions* — what the registry does with an event, where a task lands, whether a quorum agrees — live in cores that P4 doesn't touch. So swapping etcd for FoundationDB at a million machines is an implementation change behind an interface, not a redesign. The clearest proof: the registry's `apply / snapshot` core is byte-for-byte the P0 code.



The store is a shell detail. Because the registry core only ever returned a next-state value and never opened a connection, its persistence sits behind a Store interface — so etcd → FoundationDB is a shell swap, and every registry decision test still passes untouched.

02 Components

Sharded registry

REGISTRY · DELTA FROM P0

Responsibility — the same registry, now on FoundationDB and partitioned by key range so no single store sees the whole fleet. P0's `apply` / `snapshot` is unchanged; only the store shell and one pure partitioning function are new.

◆ FUNCTIONAL CORE · PURE

```
// P0 unchanged: apply(reg, ev), snapshot(reg)
func shardOf(key Key) ShardID
    // range partition — which shard owns a key
```

▮ IMPERATIVE SHELL · I/O

- FoundationDB transactions behind the `Store` interface
- route each key to its shard via `shardOf`
- replication + failover are the store's job, not ours

Boundary — a key in; a shard out. The FCIS proof of the phase: going to FDB added one pure function and a new shell, and changed no decision logic anywhere.

Observability aggregation

OBSERVABILITY · NEW

Responsibility — keep metric cardinality bounded at a million machines by rolling up per cell, then per region, then global (brief rule O1). Pure associative folds; only collection and emission are I/O.

◆ FUNCTIONAL CORE · PURE

```
func rollup(cells []CellMetrics) RegionMetrics
func rollup(rs []RegionMetrics) GlobalMetrics
func budget(level Level) Cardinality
// how much detail each level keeps
```

▮ IMPERATIVE SHELL · I/O

- receive metrics from agents; emit via OpenTelemetry
- store only the reduced series

Boundary — raw metrics in; a bounded rollup out. Because `rollup` is associative, the two-step hierarchical reduce equals a flat one — a property test, and the reason cardinality stays bounded instead of exploding with fleet size.

Backpressure & rate-limiting

TRANSPORT · NEW

Responsibility — keep the control plane stable under load by deciding, purely, when to admit, throttle, or shed. Enforcement is the shell's job.

◆ FUNCTIONAL CORE · PURE

```
func admitUnderLoad(req Req, load LoadState,
                    lim Limits) LoadDecision
    // = Admit | Throttle{delay} | Shed
func updateLoad(load LoadState, ev LoadEvent) LoadState
```

▮ IMPERATIVE SHELL · I/O

- measure queue depths and in-flight RPCs
- apply the decision: delay, reject, or token-bucket

Boundary — request + load snapshot + limits in; a decision out. "At 95% capacity, low-priority → Shed" is a pure test, so the shell never has to guess when to push back.

Rolling upgrade

DEPLOY · NEW

Responsibility — upgrade the fleet one cell at a time with zero job loss, only across compatible versions (brief rule O2). The sequencing and skew-safety are pure; the cordon/roll is shell.

◆ FUNCTIONAL CORE · PURE

```
func nextDrain(fleet FleetState, plan UpgradePlan) DrainStep
  // DrainStep = Cordon{CellID} | Done
func skewSafe(a, b Version) bool
  // is this version pair wire-compatible?
```

▮ IMPERATIVE SHELL · I/O

- cordon the chosen cell; let jobs checkpoint-migrate or drain
- roll the binary; uncordon
- refuse a step if `skewSafe` is false

Boundary — fleet + plan in; a drain step out. The upgrade is a pure plan the shell executes; "never drain two cells of the same job at once" is a test on `nextDrain`, so zero-loss is a decision, not a hope.

03 The 1M-node validation targets the shell

Because the decision cores were exhaustively tested in isolation across P0–P3, the sustained-scale run isn't hunting logic bugs — it's proving the **infrastructure** holds. That's a cleaner test: any failure is a shell/scale problem (a store limit, a transport ceiling), not a tangled question of "is the logic also wrong?"

- ✓ **Registry write rate** — FoundationDB sustains membership churn across the sharded key space.

✓ **Metric cardinality** — rollups keep the observability series bounded as machine count climbs.

✓ **Control-plane stability** — backpressure holds admission steady under a load spike.

✓ **Zero-loss upgrade** — a full rolling upgrade completes with no dropped jobs and tolerated version skew.

The boundary made the scale test tractable. With cores proven pure and the shells the only variable, "does it work at a million?" becomes a set of infrastructure SLOs to measure — not a search for correctness bugs hiding in the interaction of logic and I/O.

04 What P4 defers

P4 makes the swarm survive its own size. Two phases remain, both over the same boundary. **P5 (production readiness)** adds multi-tenancy (quotas, fairness, isolation), disaster recovery + chaos testing, SLAs, and optional core-tier attestation — new admission and policy *cores* with hardening in the shell. **P6 (self-tuning)** upgrades mitosis to split on a measured signal and refines capability/locality placement (short of a solver, per brief §18) — the last cores to write, still returning decisions the shell carries out.